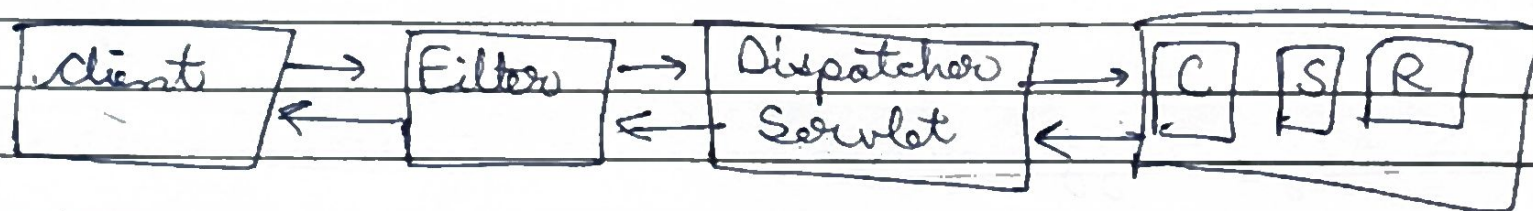


Date.....

## \* Spring Boot Filter :-

A Filter is a component that executes common logic on incoming HTTP requests before they reach the DispatcherServlet and on the outgoing responses before they are sent back to the client.



## \* Why do we need them?

Imagine you have 50 different APIs. If you need to perform repetitive tasks like logging, authentication or tracking execution time, you wouldn't want to copy paste that code into every single controller. This leads to "tight coupling" and violates Single Responsibility Principle.

## \* The Request - Response Flow :-

Incoming Request :- Client → Filter → DS → Controller → Service → Repository

Outgoing Response :- Repo → Service → Controller → DS → Filter → Client

## \* Implementing a Custom Filter :-

To create a custom filter in Spring Boot, you need to implement the 'Filter' interface.

This interface requires you to override the `doFilter()` method, which is where your core logic resides.

### → doFilter() Method :-

This method acts as a gateway for requests. It takes three parameters:

1. ServletRequest → The incoming request from the client.
2. ServletResponse → The response being built.
3. FilterChain → The object used to pass the request to next filter or final destination (`DispatcherServlet`).

### → Steps for Implementation :-

1. Creates a class → Create a class and annotate it with `@Component` so that Spring manages it as a bean.
2. Implement Interface → Implement `javax.servlet.Filter`.

Date.....

3. Define Logic → Inside `doFilter()`, write your pre-processing code.

4. Use FilterChain → Call `filterChain.doFilter(request, response)` to let the request continue to next layer. If you don't call this, the request is blocked and never reaches the controller.

5. Post-processing → Any code written after the `filterChain.doFilter()` call will execute on the way back (as the response is being returned).

@Component

```
public class MyFilter implements Filter {  
    public void doFilter(ServletRequest req, ServletResponse  
        res, FilterChain chain) {
```

//1. Pre-processing logic

```
        S.out.println("Request entering filter");
```

//2. Pass the request forward

```
        chain.doFilter(req, res);
```

//3. Post-processing logic

```
        S.out.println("Request exiting filter");
```

```
    }
```

```
}
```

## \* Filter Chaining and Order of Execution

When you have multiple filters in your application, they form a Filter Chain. It is crucial to manage the order in which these filters execute to ensure that security checks, logging, and other cross-cutting concerns happen in the correct sequence.

### → Role of @Order

If you do not specify an order, Spring Boot will decide the execution sequence based on when the beans are created, which is unpredictable. To gain full control, you can use the `@Order` annotation.

- `@Order(1)` → Assigned to the first filter (e.g. an Authentication Filter). This ensures that unauthorized requests are blocked before they reach any further processing.
- `@Order(2)` → Assigned to second filter (e.g. a Logging Filter). This runs only after the request has passed the initial security checks.

## → Key Principles for Best Practices:-

1. Separation of Concerns: Keep your filters specialized. An Authentication Filter should only handle security, while a Logging Filter should only handle request/response metadata. Do not create "Master Filter".

2. Blocking Request: You can block a request within a filter by setting an error status and returning before calling `filterChain.doFilter()`.

3. Last Processing: Always use a "finally" block or code after `chain.doFilter()` to handle tasks that must run regardless of the outcome, such as calculating API response time.

## \* Modifying Response Headers & Status Codes:-

To modify a response, you must cast the generic `ServletResponse` to `HttpServletResponse`. It is crucial to perform modifications before calling `chain.doFilter()` if you are altering the response headers or setting a status code based on authentication.

```
httpResponse.setHeader("X-Request-ID", "12345");
chain.doFilter(request, response);
```

Date.....

→ Why Wrapper Classes are Necessary?

Normally, the request/response body is an `InputStream` that can only be read once. If you need to modify the response body, the original `HttpServletResponse` is often "non-editable".

ContentCachingResponseWrapper: This is a utility used to cache the response content, allowing you to read it, modify it, and then re-write it back to client using 'copyBodyToResponse()'.  
Sagar 02/4

Note: You can modify bodies using wrappers, it is often considered bad practice as it breaks client expectations and MVC contract. Always prefer handling response logic in Service or Controller layers.

\* Working with Request Bodies & Input Streams

→ Why Request Body is "Read-Once"?

Because HTTP request bodies can contain diverse data types (JSON, files, audio, etc.). Tomcat container treats them as a generic stream of bytes. When you call

Date.....

'request.getInputStream()' or 'request.getReader()',  
the stream is consumed. Once read, it  
cannot be read again by DispatcherServlet  
to map the data to your Controller's DTO,  
resulting in "500 Internal Server Error".

### → Safely Reading & Modifying Request Bodies

To log or modify a request body without  
breaking the downstream flow, you must  
use a wrapper:-

HttpServletRequestWrapper → By extending this class, you  
can cache the byte array of  
request body.

Strategy → Read the input stream once and  
store bytes in cache.

Override `getInputStream()` & `getReader()`  
in your wrapper to return a new stream  
from cached data.

Pass your custom RequestWrapper into the  
`chain.doFilter()` method instead of the  
original request body.

Note:- Just like response modification, altering  
the request body is bad practice &  
discouraged because it breaks  
API contracts.

\* Filter Registration:- Instead of solely relying on `@Component` for automatic registration, you can use a configuration class with 'FilterRegistrationBean' to gain granular control.

→ Why use it? → It allows you to customize the filter's behaviour without modifying filter's class itself.

→ Capabilities:-

- `setOrdered()` → Explicitly define the execution sequence of your filters.
- `addUrlPatterns()` → Restrict the filter to run only on specific endpoints.
- `setName()` → Assign a logical name to your filter for easier debugging & logging.

\* Conditional Filtering:-

Sometimes you want to skip a filter entirely for some paths. You have two options:-

- Manual Check → Inside your `doFilter` method, check the request URI and call `chain.doFilter()` immediately if it doesn't match your criteria.

Date.....

- OncePerRequest Filter → Spring provides this base class to ensure a filter is only executed once per request. It also includes an overridable method called "shouldNotFilter".

In this method, you can return true for paths where filter is unnecessary.